

Classification of Mathematical Problems

Kyle Chen Deniz Acar Köstem Ethan Tan

New York University

{kc5371, dk4788, et2434}@nyu.edu

Abstract

Large language models (LLMs) show strong performance on mathematical problem solving, but remain highly sensitive to prompt design. We introduce a system that classifies math problems into interpretable tags—*problem type*, *topic*, and *method*—to support downstream uses such as tailored LLM prompt routing. We construct a small corpus of math problems annotated with these labels and compare BERT with CharCNN enriched with symbol-aware features such as counts of LaTeX operators ($\int$, $\sum$, $\lim$). We evaluate classification performance using accuracy and macro-F1, with particular focus on method prediction as a proxy for potential routing feasibility in such systems. Results show that CharCNN with symbol-aware features substantially outperforms BERT on method classification (56.5% vs. 45.5% on 1000 samples), though much of this predictive signal comes from coarse-grained task metadata rather than symbolic features.

1 Introduction

Large language models (LLMs) have achieved impressive results on mathematical problem solving and reasoning, yet their performance varies significantly based on prompt design. Different problem types expose different failure modes: proof-oriented prompts may produce well-structured arguments that cite non-existent theorems or misapply definitions, while computation-focused prompts may invoke invalid techniques or fabricate mathematical results. Rather than relying on a single prompting strategy for all problems, we propose a classification-based routing system. First identify the problem’s *type*, *topic*, and *method*. Then select a tailored prompt matched to these characteristics.

Given a math problem, our system predicts three labels:

1. **problem type** (e.g., compute, proof, conceptual),
2. **topic** (e.g., algebra, calculus, probability),
3. **method** (e.g., induction, contradiction).

We construct a new corpus with these annotations and compare BERT with a CharCNN model that combines contextual representations with symbolic cues similar to Zhang et al. (2015). Intrinsic evaluation in this work measures classification quality. A natural next step is extrinsic evaluation, where method prediction would be used to condition downstream LLM solvers and measure any resulting gains in problem solving accuracy. If the classifier can reliably identify the method, it could in principle route each problem to an appropriate prompt.

Our contributions:

- A new annotated corpus for math problem tagging.
- A symbol-aware neural classifier for multi-label prediction.
- Intrinsic evaluation demonstrating the benefits of symbolic features for math problem classification, with method prediction serving as a proxy for potential routing feasibility in LLM-based systems.

2 Related Work

Our work draws on three areas: transformer-based text classification, character-level modeling, and mathematical problem solving.

Pre-trained models such as BERT (Devlin et al., 2019) provide strong baselines for text classification. We fine-tune bert-base-uncased with separate heads for each math-specific label set.

Character-level models such as CharCNN (Zhang et al., 2015) avoid tokenization errors and are well-suited for symbol-heavy text. We implement a CharCNN encoder—following the

character-level modeling strategy of Zhang et al. (2015)—and augment it with explicit symbol features that track LaTeX operators and mathematical keywords.

Recent work on mathematical problem classification highlights the challenges of modeling symbol-heavy text. Lao and Lei (2023) study subject classification and difficulty ranking of math problems using LSTM and BERT architectures, and find that lightweight recurrent models benefit substantially from feature engineering while transformer models perform best when symbolic structure is preserved. Although their task differs from ours, their findings reinforce the intuition that mathematical text behaves differently from natural language data. Symbolic elements such as operators, variable names, and LaTeX expressions introduce irregular patterns that standard tokenizers struggle to represent, which motivates character-level or symbol-aware modeling. Our CharCNN approach builds on similar insights by extending character-level representations with handcrafted symbolic features tailored to mathematical structure. Their results thus provide conceptual grounding for our design choices, even though our focus is on type, topic, and method classification rather than subject labeling.

3 Methodology

3.1 Problem Formulation

We perform multi-task classification over mathematical problems. Given raw problem text, the model predicts: (1) problem type, (2) topic, (3) method, and (4) difficulty. In practice we treat the task as four parallel single-label tasks that can be trained jointly.

3.2 Dataset

The dataset contains 1000 mathematical problems spanning multiple difficulty levels and topics from secondary to early undergraduate curricula. Each problem is annotated with four labels:

- **Problem Type** (3 classes): compute, proof, reasoning
- **Topic** (7 classes): Algebra, Geometry, Calculus, Probability & Statistics, Linear Algebra, Pre-Calculus, Discrete Mathematics
- **Method** (233 subtopic classes): e.g., linear_equation, quadratic, derivative, basic_probability

- **Difficulty** (3 classes): easy, medium, hard

The problems cover a broad range of mathematical maturity, from basic algebra and geometry (e.g., solving linear equations, computing areas of simple shapes) to intermediate material (systems of equations, trigonometry, introductory calculus) and more advanced topics (linear transformations, induction, Bayes’ theorem). Difficulty labels reflect relative complexity within each topic. We use an 80/20 train-validation split with approximately balanced label distributions across all four tasks.

3.3 Preprocessing

We start from a raw CSV file of math problems and metadata. Each row contains a free-text problem statement (problem) and four categorical labels: category (topic), subtopic (method), difficulty, and problem type.

Data cleaning and label encoding We drop rows missing any of the four label columns and normalize whitespace in each label field by converting entries to strings, stripping leading and trailing spaces, and collapsing multiple internal spaces into a single space.

Each categorical column is mapped to an integer ID using LabelEncoder, producing `topic_id`, `method_id`, `difficulty_id`, and `problem_type_id`. These define the prediction spaces for the four tasks, with class counts N_{topic} , N_{method} , $N_{\text{difficulty}}$, and N_{ptype} . The original problem column is renamed to `text`, and the cleaned table is stored as `math_dataset_clean.csv`.

Train and validation split We construct an 80/20 train and validation split using stratified sampling on `problem_type_id` to preserve the distribution of problem types. The resulting splits are saved as `train.csv` and `val.csv`.

BERT baseline input encoding For the BERT baseline, a `MathProblemDataset` reads from the CSV files and uses a HuggingFace `AutoTokenizer` initialized with `bert-base-uncased`. Each problem statement text is tokenized with `WordPiece`, truncated or padded to a maximum length of 256 tokens, and converted into `input_ids` and `attention_mask`. Each example is paired with the four integer labels for multi-task training.

CharCNN model input encoding Our main model is a character-level CNN (CharCNN) clas-

sifier. A separate `MathProblemCharDataset` uses the same cleaned CSV files and label IDs but replaces subword tokenization with character-level encoding.

We build a character vocabulary over the first 128 ASCII symbols and add special tokens `<PAD>` and `<UNK>`. Each problem statement is mapped to a sequence of character indices of fixed length 512. Out-of-vocabulary characters are mapped to `<UNK>`, and shorter sequences are padded with `<PAD>`. A custom collate function stacks the `char_ids` tensors into batches and also passes the raw text for computing additional symbol features.

In summary, both the BERT baseline and the CharCNN model share the same cleaned, label-encoded dataset and four supervised tasks. The only difference in preprocessing is the input encoding. WordPiece tokens of length up to 256 for the BERT baseline, and fixed-length character sequences of length 512 plus auxiliary symbol features for CharCNN.

3.4 Models

3.4.1 BERT Baselines

We use `bert-base-uncased` as a text encoder and train *separate* single-task classifiers for each label. For each task $y \in \{\text{problem type, topic, method, difficulty}\}$ we fine-tune an independent `AutoModelForSequenceClassification` model on the problem text.

Training configuration Batch size: 8. Optimizer: AdamW. Learning rate: 2×10^{-5} . Scheduler: linear decay without warmup. Epochs: 10. Loss: cross-entropy for a single label per model.

BERT with auxiliary labels for method prediction In a second setting we focus on predicting the *method* label and treat the other three labels as auxiliary features. We replace the single-task method classifier with a `BERTWithAuxiliaryFeatures` model that: (1) encodes the problem text with `bert-base-uncased`, (2) embeds problem type, topic, and difficulty via three 16-dimensional embedding layers, (3) concatenates the pooled BERT output (768d) with the three label embeddings (48d total), and (4) feeds the resulting 816-dimensional vector into a linear classifier over method classes. Optimization hyperparameters are the same as above. This model is evaluated only on method prediction.

3.4.2 CharCNN with symbol features

Our main model is a character-level CNN with auxiliary symbol features and a shared multi-task head. It consists of: (1) character embeddings of dimension 64 for a 128-symbol ASCII vocabulary, (2) three 1D convolution layers with kernel sizes 3, 5, and 7 and 256 filters each, (3) batch normalization and max pooling over the sequence for each convolutional branch, (4) a 10-dimensional hand-crafted symbol-feature vector per problem (LaTeX operators, proof vs. compute keywords, equality counts, text length, calculus and linear algebra indicators), projected to 64 dimensions, (5) a shared feed-forward encoder (512 then 256 units with ReLU, batch normalization, and dropout), and (6) four task-specific classification heads for problem type, topic, method, and difficulty.

Training configuration Batch size: 32. Optimizer: Adam. Learning rate: 0.001. Scheduler: ReduceLROnPlateau on validation average accuracy. Epochs: 20. Loss: sum of four cross-entropy losses (equal weight per task).

CharCNN with auxiliary labels for method prediction For the method-only experiment we use a variant `CharCNNWithAuxiliaryFeatures`. It shares the same character convolutions and symbol features, but adds three 16-dimensional embeddings for problem type, topic, and difficulty, concatenates CNN features, projected symbol features, and the three label embeddings, and passes the combined vector through the shared encoder and a single method head. Hyperparameters match the multi-task CharCNN, but the loss is computed only on the method label.

3.5 Evaluation Metrics

For each classification task we report: (1) **Accuracy**. Overall proportion of correctly predicted labels. (2) **Macro F1**. Unweighted mean of per-class F1 scores, highlighting performance on rare classes. (3) **Weighted F1**. Class-frequency-weighted F1, summarizing performance under label imbalance.

For the multi-task CharCNN model, we compute these metrics per task and additionally report *exact-match accuracy*, defined as the fraction of problems for which all four labels are predicted correctly simultaneously.

3.6 Experimental Setup

BERT Baseline Train four independent models. Select the best checkpoint by validation accuracy.

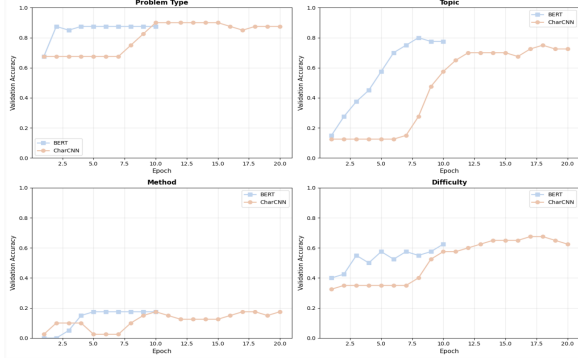


Figure 1: Validation accuracy on the 250-sample dataset with direct training from problem text. CharCNN exhibits delayed but sustained learning across all tasks, while BERT shows early peaks followed by performance decline.

CharCNN Train a single multi-task model with shared parameters. Select by mean validation accuracy across all tasks.

The classifier is intended to serve as a front-end for downstream LLM reasoning, although we focus only on intrinsic evaluation here.

4 Experiments

4.1 Impact of Dataset Size on Model Performance

We analyze learning dynamics under two data regimes to understand how dataset size affects model behavior and to validate our hypothesis that method classification benefits from character-level symbolic modeling.

4.1.1 Limited Data Regime: 250 Samples

Figure 1 shows validation accuracy over 20 epochs on a subset of 250 samples when models predict all four labels directly from problem text.

CharCNN learns more slowly at first but continues to improve throughout training, with method accuracy rising steadily rather than plateauing. BERT starts from a stronger initialization but quickly overfits the 250-sample subset, showing declining performance after early peaks. Key observations. **Method.** CharCNN reaches 18% while BERT plateaus at 17%. **Topic.** CharCNN climbs to 73%, whereas BERT peaks at 80% before declining to 77%. **Problem Type.** both models converge around 88–90%. **Difficulty.** CharCNN reaches 68%, while BERT peaks at 62% and then declines.

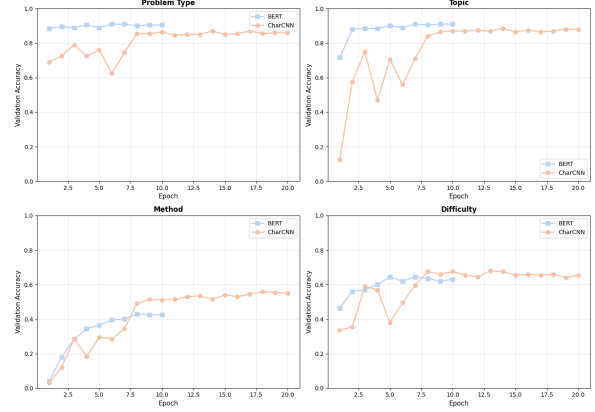


Figure 2: Validation accuracy on 1000-sample dataset with direct training from problem text. CharCNN demonstrates substantial improvements across all tasks, with particularly strong performance on method classification (56.5%).

4.1.2 Full Dataset: 1000 Samples

Figure 2 shows validation accuracy over 20 epochs when both models predict all four labels directly from problem text on the full 1000-sample dataset.

With 4 times more data, CharCNN’s method performance improves from 18% to 56.5% (a 38.5 point gain), while BERT improves from 17% to 45.5% (a 28.5 point gain), indicating that CharCNN benefits more from additional training data. Key observations. **Method.** CharCNN reaches 56.5%, BERT plateaus at 45.5%. **Topic.** CharCNN reaches 90%, BERT reaches 94%. **Problem Type.** Both converge around 88–90%. **Difficulty.** CharCNN reaches 68%, BERT peaks at 64%.

To understand whether method can be inferred from task metadata alone, we trained separate lightweight feedforward classifiers to predict method using only the predicted Type, Topic, and Difficulty labels. The partial labels approach achieves 55% (CharCNN) and 42.5% (BERT), compared to 56.5% and 45.5% for direct classification. A gap of only 1.5–3 percentage points.

4.2 Results Summary

Table 1 summarizes final classification performance across all four tasks on the 1000-sample dataset.

CharCNN demonstrates substantial advantages on Method (an absolute gain of 11 percentage points) and modest gains on Difficulty (4 percentage points), while BERT performs slightly better on Topic. Problem Type performance is comparable between architectures.

Task	Classes	BERT	CharCNN	Gain
Problem Type	3	89.0	90.0	+1.0
Topic	7	94.0	90.0	−4.0
Method	233	45.5	56.5	+11.0
Difficulty	3	64.0	68.0	+4.0

Table 1: Validation accuracy on the 1000-sample dataset (in %).

5 Discussion

Our findings connect to broader patterns observed in prior research on mathematical text classification. [Lao and Lei \(2023\)](#) report that subject-level distinctions in math problems are often confounded by difficulty, with misclassifications frequently occurring between adjacent algebra categories. This mirrors our observation that method labels are highly entangled with simpler task metadata such as problem type and topic. In both settings, classifiers appear to rely primarily on hierarchical regularities in math curricula rather than deep symbolic structure in text. At the same time, our results diverge from theirs in architectural behavior. While [Lao and Lei \(2023\)](#) find that BERT outperforms LSTM for subject classification, our experiments show that symbol-aware CharCNN surpasses BERT on method prediction. This contrast suggests that transformer advantages may diminish as label spaces become more fine-grained and symbolic density increases, which highlights the importance of model selection in symbol-centric mathematical tasks.

5.1 Symbol-Aware Features vs. Task Metadata

Our central finding challenges the intuitive assumption that mathematical reasoning classification requires deep symbolic analysis. Our partial labels analysis reveals that simple feedforward classifiers using only predicted Type, Topic, and Difficulty labels achieve 55% accuracy (compared to CharCNN’s 56.5%) and 42.5% accuracy (compared to BERT’s 45.5%). Gaps of only 1.5–3 percentage points. This minimal difference raises fundamental questions about the value of analyzing mathematical text content versus simple categorical reasoning.

The strong predictive power of task metadata suggests that method classification is largely determined by problem categorization rather than mathematical content analysis. For instance, calculus problems naturally tend toward derivative and integration methods, while algebra problems

cluster around equation-solving techniques. This indicates that our current method taxonomy may be capturing taxonomic regularities. Relationships between problem domains and typical solution approaches. Rather than requiring sophisticated analysis of mathematical symbols, notation, or textual content. The marginal benefit from direct text analysis suggests that much of what appears to be mathematical reasoning classification may actually be domain-specific pattern recognition.

5.2 Practical Implications for LLM Based Routing

From a systems perspective, our results present a nuanced picture for practical routing applications. The absolute performance levels for method prediction. 56.5% for CharCNN and 45.5% for BERT. Remain modest for practical deployment, regardless of whether we use direct text analysis or metadata-based approaches. A routing system operating at 56.5% accuracy would correctly identify the appropriate method for roughly half of incoming problems, which may be insufficient for production use.

However, these results should be interpreted within the constraints of our dataset size and scope. The continued upward trajectory of CharCNN in the limited-data regime suggests that method classification accuracy could improve substantially with larger annotated corpora. Additionally, even imperfect routing may provide value if the cost of misclassification is low. For instance, if incorrect method predictions simply result in suboptimal but still reasonable prompting strategies rather than complete failure. The small performance gap between direct classification and metadata-based prediction could translate to meaningful differences in downstream LLM solution quality, particularly for problems where method identification is most challenging.

5.3 Dataset Scale and Architectural Choices

The scaling behavior from 250 to 1000 samples reveals different learning dynamics between architectures. CharCNN shows larger improvements on method prediction (38.5 point gain) compared to BERT (28.5 point gain), suggesting that character-level models benefit more from additional training data in symbol-heavy domains. The continued upward trajectory of CharCNN in the limited-data regime indicates substantial headroom with larger annotated corpora.

This pattern suggests that current dataset constraints may be masking the true potential of symbolic modeling. With sufficient training data, the gap between symbol-aware and metadata-based approaches might widen, particularly for problems with richer mathematical notation or more complex symbolic relationships.

5.4 Methodological Limitations and Future Directions

Our approach treats method, type, topic, and difficulty as independent single-label classifications, which may not reflect the true structure of mathematical reasoning. Many problems admit multiple solution strategies or combine techniques from different areas. A multi-label formulation or hierarchical classification scheme might provide clearer separation between reasoning strategies and problem characteristics.

Additionally, our symbol-aware features are limited to simple occurrence counts and keyword matching. More sophisticated representations, such as abstract syntax trees for mathematical expressions, graph-based encodings of formula structure, or learned embeddings for symbolic components, might unlock greater benefits from character-level modeling. The dataset scope also constrains generalizability, as our corpus spans only secondary to early undergraduate mathematics.

5.5 Implications for Mathematical AI Systems

Our findings have broader implications for how we approach mathematical reasoning in AI systems. The fact that method classification is largely predictable from coarse problem characteristics suggests that current approaches to mathematical problem understanding may be more shallow than commonly assumed. Rather than identifying deep structural patterns in mathematical reasoning, our classifiers may be recognizing surface-level linguistic and domain cues.

This observation points toward a need for more sophisticated evaluation frameworks that measure genuine reasoning strategy identification rather than taxonomic classification. Future work might explore whether models can predict solution difficulty, identify multiple valid approaches, or recognize when problems require novel technique combinations. Capabilities that would be more directly useful for mathematical AI applications.

6 Conclusion

We present a classification system for mathematical problems that predicts problem type, topic, method, and difficulty. These labels are motivated by potential downstream uses such as selecting specialized solution strategies or prompt templates in LLM-based systems, although we do not implement any such routing pipeline in this work. Our key findings demonstrate that symbol-aware character-level models substantially outperform BERT on method prediction, achieving 56.5% accuracy compared to 45.5% on our 1000 problem corpus. However, a critical discovery emerges from our partial labels analysis. Simple metadata-based classifiers that predict method from Type, Topic, and Difficulty alone reach 55% accuracy for CharCNN and 42.5% for BERT, compared to 56.5% and 45.5% when using the full problem text. In other words, metadata-only models fall short by just 1.5–3 absolute percentage points.

This finding suggests that method classification in our dataset is largely driven by coarse problem characteristics rather than fine-grained symbolic structure. While CharCNN shows superior scaling behavior (38.5 point improvement vs. 28.5 points for BERT when expanding from 250 to 1000 samples), the marginal benefit of symbolic features raises important questions about how much additional value complex neural architectures actually provide for method tagging in this setting.

The continued improvement trajectory of CharCNN in limited data regimes indicates substantial potential with larger annotated corpora. However, our results highlight the need for refined method taxonomies that better separate genuine reasoning strategies from surface-level problem characteristics. Future work should explore whether more discriminative annotation schemes and richer symbolic representations can unlock the full potential of character-level modeling for mathematical reasoning classification and for downstream applications such as solver selection or prompt design.

Limitations

Our study has several limitations. First, the dataset is relatively small (1000 problems) and drawn from secondary to early undergraduate curricula, so the results may not generalize to competition-level, highly specialized, or advanced graduate mathematics. Second, we enforce single-label annotations for problem type, topic, and method even when

problems plausibly admit multiple categories or solution paths, which may collapse distinct reasoning strategies into a single tag.

Third, our symbol-aware features are simple occurrence counts of LaTeX operators and keywords and do not capture the structure of mathematical expressions, the relationships between symbols, or the steps of a solution. This limits the extent to which our CharCNN can exploit genuinely symbolic information. Fourth, the strong performance of the partial labels models (they are within 1.5–3 absolute percentage points of the direct text models on method accuracy) indicates that method is heavily entangled with problem type and topic. This suggests that our current method taxonomy may not cleanly separate reasoning strategy from coarse task metadata and that the benefits of symbol-aware modeling for downstream control are likely to be modest in this setting.

Finally, our evaluation is purely intrinsic. We measure classification quality but do not couple the classifier to any downstream solver or LLM-based system. As a result, it remains unclear how much of the observed gap between models, or between direct and partial labels prediction, would translate into end-to-end improvements in mathematical problem solving or user-facing performance.

Future Work

Future work will focus on addressing the limitations identified in our analysis and on extending the classifier toward practical routing applications.

Expanding the dataset. Both BERT and CharCNN show clear scaling effects, with both models showing particularly strong gains when moving from 250 to 1000 samples. This suggests that method classification may be data-limited rather than architecture-limited. A substantially larger and more diverse corpus, including competition problems, multi-step reasoning tasks, and symbol-heavy expressions, would allow us to evaluate whether direct text modeling can eventually outperform metadata-based prediction or whether the predictive signal will continue to be dominated by coarse task categories.

Revisiting the method taxonomy. Our partial-label experiments show that method is highly entangled with Type, Topic, and Difficulty. This indicates that the current taxonomy may not cleanly separate reasoning strategy from domain category.

Future work may develop a hierarchical or multi-label method schema that better captures the structure of mathematical reasoning (e.g., distinguishing algebraic manipulation from structural arguments such as induction or contradiction). A refined taxonomy may reveal distinctions that are currently collapsed into broad method classes.

Joint modeling of solution steps. Predicting method from problem text alone may not provide enough signal, particularly for problems whose surface form does not explicitly reveal the underlying technique. Future extensions could incorporate partial or full solution traces, sketches of solution steps, or exemplars of typical reasoning patterns. This would shift the task from inferring method from problem statements to inferring method from observed reasoning structure, potentially supporting richer models of mathematical strategy.

Integration with downstream LLM solvers.

Our work is evaluated purely on intrinsic metrics. An important extension is to embed the classifier into an end-to-end pipeline that routes each problem to a specialized prompt or solver. This would allow us to measure whether even modest method-classification accuracy produces meaningful improvements in solution quality. Such a system would also enable the study of cost–benefit trade-offs, such as whether slight misclassifications produce graceful degradation or catastrophic solver failures.

Uncertainty and confidence modeling. Robust routing requires calibrated uncertainty estimates rather than raw predicted labels. Future work could explore temperature scaling, Bayesian inference, or conformal prediction to produce confidence intervals that determine when to trust a routing decision and when to fall back to a general-purpose solver. Calibrated uncertainty is particularly important in mathematical reasoning, where incorrect method predictions may induce systematic errors in downstream solution attempts.

Acknowledgments

We would like to express our sincere appreciation to Kishlay Kumar for his guidance and support throughout this research project. We also thank Professor Adam Meyers for his feedback and encouragement during the semester.

References

- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [Bert: Pre-training of deep bidirectional transformers for language understanding](#). In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.
- Anthony W. F. Lao and Philip I. S. Lei. 2023. Subject classification and difficulty ranking of math problems. In *2023 IEEE 11th International Conference on Information, Communication and Networks (ICICN)*, pages 844–850. IEEE.
- Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*.